

Product Brief for Technical Architecture and Product Strategy

Production-Ready Rules-and-Workflow Intelligence Platform

Prepared for: Candidate Engineering Review

1. What Problem the Product Solves

Organizations operating in complex, rules-driven environments must interpret and operationalize a large volume of external requirements across intake, approval, documentation, submission, settlement, and audit workflows.

Those requirements are fragmented across portals, manuals, fee schedules, policy bulletins, and other semi-structured or unstructured sources. They are hard to locate, inconsistent in format, and frequently updated.

Most teams still translate these requirements manually into spreadsheets, internal playbooks, and staff knowledge. This creates avoidable rejections, rework, delayed throughput, compliance risk, and dependency on tribal knowledge.

2. Product Concept

The product is a software platform that converts fragmented policy and operational requirements into structured, searchable, workflow-ready intelligence.

It should help operational users answer questions such as: What rules apply, to whom, under what conditions, in which jurisdiction, for which operating scenario, and what actions or documentation are required before work is submitted or finalized?

At a high level, the system should combine multi-source ingestion, automated rule extraction and validation, normalization, versioning, search, and workflow delivery, with optional human QA for exceptions or low-confidence changes.

3. Intended Users and Primary Jobs to Be Done

User Group	Primary Job	What They Need From the Product
Intake / Front-Line Operations	Verify requirements before work is accepted or advanced	Fast answers, confidence flags, required documents, and escalation paths
Review / Approval Teams	Understand decision rules, thresholds, and renewal logic	Structured rules, source traceability, and scenario-specific differences
Submission / Execution Teams	Complete downstream steps without preventable errors	Submission methods, rule checks, exception handling, and dependencies
Operations / Compliance Leadership	Monitor changes and standardize execution across the organization	Version history, audit trail, change alerts, and governed rule ownership

4. Core Product Capabilities

The platform is organized around six core capabilities, supported by cross-cutting platform services (ingestion, normalization, automated review, versioning, search, and admin).

4.1 Rules Repository

A structured, governed repository of external requirements, organized by rule source, program variant, and jurisdiction:

- Qualification and entitlement verification rules
- Approval thresholds, limits, and renewal processes
- Covered categories, qualifying codes, and limitations
- Documentation requirements (forms, attachments, signatures, evidence)
- Submission, rejection, and dispute rules and methods
- Jurisdiction-specific and program-specific nuances

4.2 Forms, Portals, and Submission Intelligence

- Direct links to current external forms and portals
- Phone-tree pathways and call guidance where required
- Submission methods (portal, fax, structured electronic formats, paper) with source-specific formatting
- Required attachments, cover sheets, and batch-submission rules

4.3 Workflow & Decision Support

- Standardized workflows for verification, approval/re-approval, and submission
- Decision logic embedded at the point of work to reduce variability
- Guided checklists, validators, and pre-submission gates keyed to source/program/jurisdiction
- Case-level memory so a given workflow's path is consistent across handoffs

4.4 Rejection Prevention & Resolution Framework

- Mapping of rejection reason codes to root causes and prevention logic
- Source-specific rejection patterns surfaced from operational outcomes
- Upstream prevention logic embedded in intake and approval workflows
- Standardized resolution and escalation pathways, including dispute templates and timelines

4.5 Continuous Maintenance & Updates

- Ongoing updates as external rules evolve (scheduled crawls, bulletin monitoring)
- Validation through real operational outcomes fed back into the system
- Feedback loops from operational teams (flag-a-rule, dispute-a-rule)
- Version control, effective dating, and auditability

4.6 Professional Services Enablement

The platform must support a services-capable delivery model — not as a separate product, but as tooling the platform exposes so implementation, enablement, and ongoing support can be delivered efficiently:

- Implementation and onboarding tooling
- Workflow configuration and embedding into end-user systems
- Training and enablement hooks
- Outcome tracking and reporting at the organization level

4.7 Cross-Cutting Platform Capabilities

- Source ingestion (PDFs, manuals, bulletins, fee schedules, portal HTML, external APIs)
- AI-assisted rule extraction with deterministic post-processing
- Normalization into a canonical rule model
- Automated review, validation, and confidence scoring
- Optional human review console for exceptions and governance
- Versioning, lineage, and audit trail tied back to source evidence
- Search and decision-support UI + API
- Change monitoring and controlled publishing
- Admin portal for non-technical rule stewardship (source onboarding, taxonomy edits, publish approvals)
- Integration delivery (web app, embeddable widgets, REST/GraphQL API, webhooks, system-of-record connectors)

5. Product Scope for the Working Platform

Assume the target is a fully working, production-capable platform — not a narrow MVP or research prototype. The architecture and product strategy should describe a usable v1 that can ingest sources, structure rules, validate outputs automatically, expose search/API experiences, monitor change, and support controlled publishing, all while being embeddable into existing operational systems.

Core Platform Scope (v1)	Optional Features
Multi-source ingestion: portals, PDF manuals, fee schedules, bulletins, forms, admin inputs	Broad custom services workflows or one-off client-specific process layers
Canonical rule data model with source lineage and auditability	Heavy predictive analytics or benchmarking products beyond core rejection prevention
Automated extraction, validation, publish controls, and confidence scoring	Manual-review-only operating model
Search, guided query experience, workflow widgets, and delivery API	Deep bespoke integrations before core platform repeatability is proven
Version history, change monitoring, effective dating, confidence display	Expansion into unrelated domains before the core product is stable
Exception review console, admin/governance portal, RBAC, tenant isolation	Large-scale per-customer customization that forks the canonical rule model
Outcome-feedback loop from structured operational data (at minimum v1.5)	Replacement of client-side systems of record

6. Representative Rule Data Model

A normalized rule object should be flexible enough to support both human-readable guidance and machine-consumable logic. A first-pass canonical schema could include (but don't limit your thinking):

Field	Description	Example Type
rule_id	Unique stable ID for the normalized rule	string (ULID)
source_ref	Document, section, URL, page, and extraction metadata with checksum	object
rule_source	Origin entity that publishes the rule	ref → source entity
program_variant	Program identifier, program type, and variant metadata	object
jurisdiction	Geographic or regulatory jurisdiction	ISO code + taxonomy
workflow_stage	intake verification approval documentation submission rejection dispute	enum
operating_scenario	Service family, category codes, qualifiers, delivery context	object
condition_logic	Structured expression of when the rule applies	structured condition set
required_action	What the user must do (submit form X, attach Y, follow path Z)	structured action object
required_documentation	Forms, evidence, notes, signatures, attachments	list
submission_method	portal fax electronic-structured paper phone-tree + endpoint details	enum + details
timing	Filing windows, approval validity, renewal cadence, dispute deadlines	durations / date ranges
rejection_reason_map	Linked rejection-reason codes and root-cause tags this rule prevents or addresses	list of refs
exceptions	Jurisdiction-, program-, or scenario-specific caveats	list
confidence	Extraction confidence (0–1) and method provenance	numeric + enum
review_status	draft auto_validated in_review approved published retired	enum
effective_dates	Start, end, superseded_by, superseded_rule_id	date range + refs
lineage	Extraction pipeline run_id, model/prompt version, reviewer IDs	object

7. Functional Requirements (not exhaustive)

- Ingest new and updated source materials (PDFs, portals, bulletins, fee schedules, external APIs, admin inputs) on scheduled and on-demand cadences.
- Extract candidate rules with full traceability to the original source passage (page, offset, snippet, checksum).
- Automate validation: schema checks, cross-rule conflict/duplication detection, confidence scoring, and publish-readiness gates.
- Optionally route low-confidence or high-impact changes to a human reviewer console to accept, edit, merge, split, reject, and publish.
- Support full-text and structured search across source, program variant, jurisdiction, workflow stage, and operating scenario.
- Display concise operational guidance with direct source references, effective dates, and confidence indicators.
- Track version history and expose what changed, when, why, and by whom.
- Trigger update workflows when source changes are detected; support automated publish paths when thresholds are met.
- Provide RBAC for editors, reviewers, administrators, and read-only end users, with multi-tenant isolation for client organizations.
- Expose an API layer so third-party systems can request normalized guidance, validate a prospective submission, or subscribe to changes.
- Ingest structured operational-outcome data from clients and link rejections back to rules (closed-loop feedback).
- Provide tooling to support implementation, onboarding, and outcome tracking at the organization level.

8. Non-Functional Requirements (not exhaustive)

- Security: role-based access, encryption in transit (TLS 1.2+) and at rest (AES-256), audit logging, secret management (KMS/Vault), and environment separation (dev/stage/prod).
- Compliance readiness: architecture must accommodate future handling of sensitive or regulated data (e.g., data processor agreements, minimum-necessary access, logging of sensitive-data access) without major rework.
- Reliability: clear ingestion retry semantics, dead-letter queues, observability (metrics, logs, traces), error handling, and recoverability for extraction pipelines. Target 99.9% API uptime.
- Scalability: incremental expansion in sources, programs, jurisdictions, rules, tenants, and end users without re-architecture; horizontal scaling for ingestion and search.
- Explainability: every rule answer must show the source evidence (document, snippet, effective date) and confidence.
- Governance: publishing supports automated policy controls, optional human approval paths, and a defensible audit trail.
- Extensibility: data model and service boundaries support future modules without a rewrite.

- Performance: p95 search latency < 500ms; rule-lookup API p95 < 300ms; ingestion-to-publish turnaround < 24 hours for priority sources.

9. Technical Requirements to Implement the Core Capabilities

This section maps each core capability to the concrete technical components needed to deliver it. It is intended to guide the architecture-options memo, system diagram, and build sequence requested in Section 11.

9.1 Rules Repository

Layer	Technical Requirements
Data Model	Canonical rule schema (see Section 6) stored as JSON documents with strict validation Separate entities: rule_source, program_variant, jurisdiction, operating_scenario, rule, source_document, rejection_reason Reference data for standard taxonomies (category codes, rejection reason codes) seeded from authoritative registries Bi-temporal versioning (valid_time + system_time)
Storage	Primary store: PostgreSQL (relational) with JSONB for flexible rule fields — or a document database if the team prefers document-native Object storage (S3-compatible) for source artifacts: PDFs, HTML snapshots, screenshots Search index: OpenSearch or Elasticsearch for full-text + structured filters Vector store (pgvector, Qdrant, or Weaviate) for semantic retrieval across source snippets
Services	Rule CRUD service with schema validation (Pydantic / Zod / JSON Schema) Publishing service with state machine: draft → auto_validated → approved → published → retired Read API with source/program/jurisdiction/scenario filters and effective-date awareness
Why These Choices	PostgreSQL + JSONB gives strong consistency and flexibility without forcing a document DB Separate object store keeps large binaries out of the relational tier Dual search (keyword + vector) supports both operator lookups and AI-assisted Q&A

9.2 Forms, Portals, and Submission Intelligence

Layer	Technical Requirements
Ingestion	Headless-browser scraping stack (Playwright or Puppeteer) for portal HTML and form metadata PDF form field extraction (pdfplumber, PyMuPDF, or a commercial equivalent) Link-health monitor that detects dead URLs, moved forms, and login-walled pages Captcha/MFA handling strategy: human-assisted queue; never store external credentials without contractual coverage and encryption

Layer	Technical Requirements
Enrichment	Form-field classifier (LLM + rules) that tags each field with purpose Submission-method taxonomy: portal fax electronic-structured paper phone-tree Phone-tree scripts represented as directed graphs (nodes = prompts, edges = input options)
Delivery	“How to submit” API returns the ranked submission path for a given (source, program variant, jurisdiction, transaction type) Embeddable widget (React + Web Components) that renders the submission checklist inside client operational tools Webhook notifications when a form version or portal URL changes
Ops	Scheduled crawls (Airflow, Temporal, or Prefect) with per-source throttling and politeness policies Crawl isolation: per-tenant proxy pools only where contractually allowed; otherwise shared egress

9.3 Workflow & Decision Support

Layer	Technical Requirements
Rule Engine	JSON-Logic or CEL (Google Common Expression Language) for condition evaluation — deterministic, auditable, portable Rule compiler that validates rules before publish (no undefined variables, type checks) Decision explainer that returns which conditions fired and which source rule(s) applied
Workflow Orchestration	Stateful workflow service (Temporal or AWS Step Functions) for multi-step flows: verification → approval → submission Case object keyed by (tenant, subject, episode) with audit trail Idempotent step execution so retries are safe
UI/UX	Guided checklist UI (React + TypeScript, design system component library) Inline decision-support panel surfaced inside the client system of record via iframe/widget/SDK Role-aware views (intake vs. approval vs. submission) driven by RBAC claims
Integration	REST + Webhooks for most clients; GraphQL where partners want flexible queries Industry-standard embedded-launch frameworks for system-of-record integration (longer-term) SDKs in JavaScript/TypeScript and Python

9.4 Rejection Prevention & Resolution Framework

Layer	Technical Requirements
Data	Ingest structured operational-outcome data (standard B2B electronic formats where available; custom parsers otherwise) Normalize rejection reason codes against published reference data Link rejections to submissions, and submissions to rules that applied (or should have applied) at submission time
Analytics	Pattern detection job that clusters rejections by (source, program variant, reason code, root cause) and surfaces emerging trends Rule-coverage report: for each rejection, did we have a rule that would have prevented it? If yes, did the workflow enforce it? Alerting when a rejection pattern crosses a configurable threshold
Loopback	Feedback API so an outcome can mark a rule as “validated,” “insufficient,” or “incorrect” Reviewer queue for rule gaps — rejections without a matching rule Dispute template service with source-specific cover letters and evidence checklists

9.5 Continuous Maintenance & Updates

Layer	Technical Requirements
Change Detection	Content-diffing pipeline: checksum + semantic diff on source documents (page-level hash, embedding distance) Structured feed listeners for sources that publish change logs or RSS/JSON Impact analyzer: which rules reference the changed passage, which tenants are affected
Extraction Pipeline	Hybrid extraction: deterministic parsers (PDF text, tables via Camelot/Tabula) + LLM extraction with structured output (JSON Schema / function-calling) LLM provider strategy: pluggable abstraction; default to a cost-efficient model for bulk extraction, escalate to larger models for ambiguous passages Self-host fallback: support open-weights models via vLLM on GPU nodes for on-prem/private-cloud deployments Prompt + model versioning tied to every rule lineage record
Cost Control for AI	Aggressive caching of extraction results keyed on (source_checksum, prompt_version, model_version) Chunking and retrieval-first design so the LLM only sees relevant passages Batch inference and off-peak scheduling; token-budget guardrails per run Quality gate: route to human review only when confidence < threshold
Regression Control	Golden-set of labeled rules per source; accuracy/precision/recall tracked per pipeline release

Layer	Technical Requirements
	A/B evaluation harness comparing prompt/model versions on the golden set Canary publishing: promote extractions to a subset of tenants before full rollout

9.6 Professional Services Enablement

Layer	Technical Requirements
Client Lifecycle	Multi-tenant model with per-tenant source/program scoping, workflow configuration, and branded UI Implementation console: checklist of onboarding, integration steps, and enablement milestones Role templates for common organization shapes (single-site, multi-site, enterprise)
Outcome Reporting	Baseline and current KPIs per tenant (configurable metric set) Executive-ready dashboard (embedded BI: Metabase/Superset, or custom Recharts views) Exportable report packs generated from templated reports
Enablement	In-app guidance (walkthroughs, tours) tied to role Training content hooks or SCORM export for formal training programs Change-management notifications for rule updates affecting a tenant's active workflows

9.7 Cross-Cutting Technical Requirements

Domain	Technical Requirements
Architecture Style	Modular monolith for v1 (single deployable, clear module boundaries: ingestion, extraction, rules, search, workflow, admin) Extract to services once load or team boundaries justify it (ingestion + extraction are the most likely first extractions) Async backbone: message bus (Kafka, NATS, or cloud-managed equivalent) for ingestion and rule-change events
Runtime & Languages	Backend: Python (FastAPI) for extraction/NLP workloads; TypeScript (NestJS/Node) or Go acceptable for core API if team prefers Frontend: React + TypeScript, design system (shadcn/ui or Material UI), Tailwind Workflows: Temporal (preferred) or Prefect/Airflow for scheduled ingestion
AI/LLM Stack	Abstraction layer (LiteLLM or custom) so model vendor can be swapped; support commercial and self-hosted open-weights Structured output via function-calling / JSON-mode; validate against rule schema before persisting Eval harness (promptfoo, Ragas, or custom) wired into CI

Domain	Technical Requirements
Data Infra	PostgreSQL 15+ (primary), Redis (cache / queues), OpenSearch (search), pgvector or Qdrant (semantic), S3-compatible object store (MinIO on-prem) ETL via dbt for reporting marts; keep transactional and analytical concerns separated Backup/restore: PITR for PostgreSQL; cross-region snapshots for object store
Deployment	Containerized (Docker) and orchestrated on Kubernetes (EKS/GKE/AKS) or Nomad for on-prem Cloud-agnostic baseline: everything deployable to AWS, GCP, Azure, or on-prem Kubernetes with Helm charts IaC with Terraform; secrets via Vault or cloud KMS CI/CD via GitHub Actions or GitLab CI with progressive delivery (canary + feature flags)
Security & Compliance	SSO (SAML/OIDC), SCIM for enterprise tenants, MFA for privileged roles RBAC + ABAC for tenant and source scoping; row-level security in PostgreSQL Sensitive-data-ready logging, contractual coverage with all subprocessors, encryption at rest/in-transit SOC 2 readiness path from day one: control inventory, evidence collection automation
Observability	Metrics (Prometheus/Grafana or Datadog), logs (Loki/ELK), traces (OpenTelemetry) Business-level dashboards: rules published/day, extraction accuracy, rejection-rule coverage, API latency per tenant Alerting with on-call rotation (PagerDuty/Opsgenie)
Self-Hosted / Private-Cloud Option	All components must be deployable without a specific cloud dependency (use portable primitives: PostgreSQL, S3 API, Kubernetes) Replace managed services with open-source equivalents (MinIO for S3, self-hosted OpenSearch, vLLM for LLM inference) Expect reduced elasticity and increased ops burden; document the trade-offs in the architecture memo

10. Architecture Questions You Should Answer

- What system architecture is most appropriate for a production-ready v1 that can scale without unnecessary early complexity: modular monolith, service-oriented design, or another approach — and why?
- How should ingestion, extraction, normalization, review, search, workflow, and delivery be separated into modules or services?
- What mix of deterministic parsing, retrieval, ML classifiers, and LLMs is appropriate for high-trust rule extraction?
- How will you control AI/LLM cost at scale (caching, chunking, model routing, batching, self-hosted options) while maintaining extraction quality?
- If a client requires self-hosted or private-cloud deployment (no AWS/GCP/Azure), what changes in the architecture, and what trade-offs do you accept?
- What datastore pattern best supports source documents, normalized rules, version history, outcome linkage, and search?
- How will change detection work across heterogeneous sources (PDF manuals, portal HTML, external APIs, admin-entered overrides)?
- What automated validation, confidence thresholds, regression tests, and exception-routing workflow are required so human review remains optional rather than mandatory?
- What observability, QA, and evaluation strategy is needed for extraction accuracy and regression control?
- What API design (REST vs. GraphQL, webhook model, embeddable widget strategy, SDKs) is best for future integration into external operational systems?
- What security, tenancy, and data-governance posture should be in place from day one?
- How does the operational-outcome feedback loop fit into the architecture without creating sensitive-data sprawl?

11. Requested Deliverables

Deliverable	What It Should Include
Architecture Options Memo	3–4 viable architecture approaches (e.g., modular monolith on managed cloud; polyglot microservices; cloud-agnostic/self-hosted; AI-first extraction-heavy vs. deterministic-first); trade-offs, recommended path, and reasoning. Include concrete product/library choices (databases, search, LLM providers, workflow engine, frontend stack, IaC).
High-Level System Diagram	Core components, data flows, trust boundaries, automation paths, ingestion and publishing pipelines, outcome-loopback, and internal vs. external interfaces.
High-Level Implementation Roadmap & Timeline	Practical build sequence for a working platform, major milestones (foundations → ingestion → extraction → rules + search → workflow embedding → outcome loopback → services tooling), and dependencies.
Data & Validation Strategy	Source ingestion approach, canonical rule schema, validation logic, confidence thresholds, exception handling, and the outcome-feedback loop.
Risk Register & Build-vs-Buy View	Top technical, operational, and data-quality risks (extraction accuracy, portal brittleness, sensitive-data exposure, LLM cost drift, tenant isolation), with which components should likely be custom-built vs. sourced.
AI/LLM Strategy Note	Model routing, cost controls, evaluation harness, self-hosted fallback, and a plan for keeping hallucinations out of published rules.

12. Closing Guidance

Treat this document as a bounded briefing. The task is not to recreate a full business plan and not to spend months rediscovering the problem. The task is to turn this brief into a pragmatic architecture and product strategy for a production-ready rules-and-workflow intelligence platform, with automation as the default operating model and optional human review reserved for exceptions, governance, and QA.

Your deliverables will help us decide if we will bring you on for an internship this summer.